

FIT3143 • PARALLEL COMPUTING

Finding Prime Numbers Using Threads & OpenMP

Erwyna Soo Wen Xin • 36555789

Taabish Farooq Bhat • 35473932



SEMESTER 2 / 2026

Problem Overview and Baseline

664,579

primes below 10 million

- Only test divisors up to the square root of k
- If $k = a \text{ times } b$, both cannot exceed the square root
- Handle 2 separately, then step by 2 to skip evens
- Flag array, one byte per number, already in order
- Timer stops before the file write

```
limit = (long)sqrt((double)k);  
for (d = 3; d <= limit; d += 2)  
    if (k % d == 0) return 0;  
return 1;
```

1 byte / number → ascending output
No sorting step required

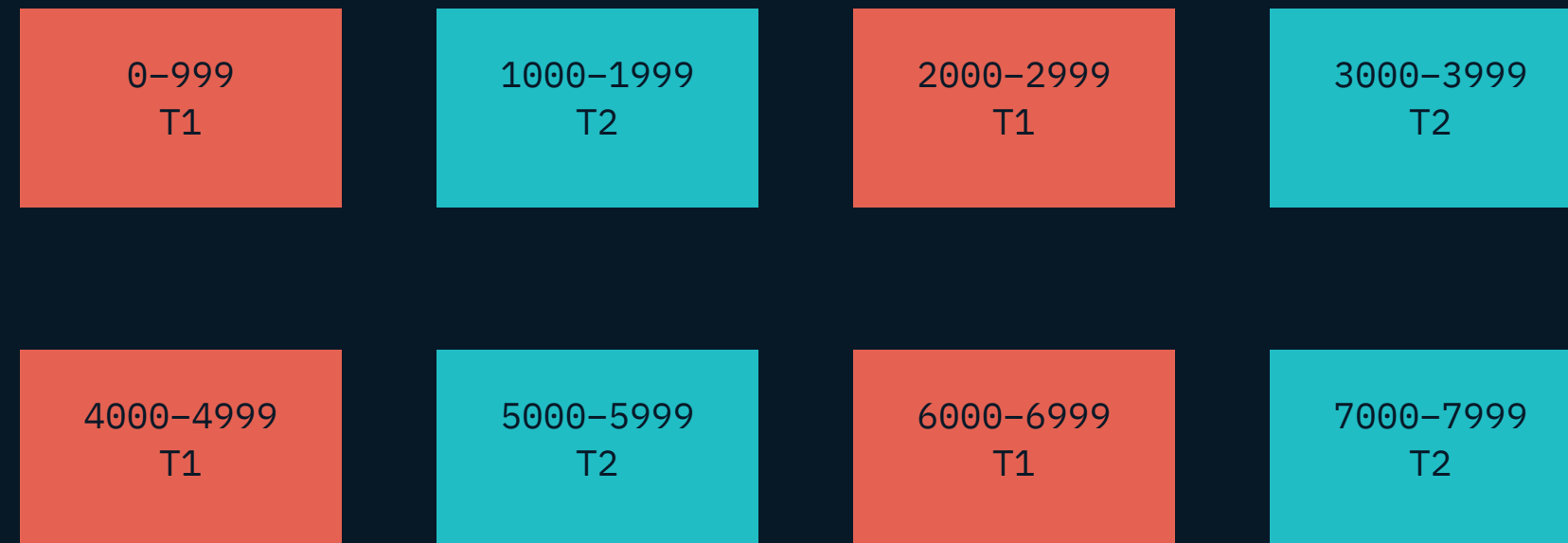
POSIX Threads, Splitting the Work

8.70x

one block each

9.75x

chunks of 1000 dealt out in rotation



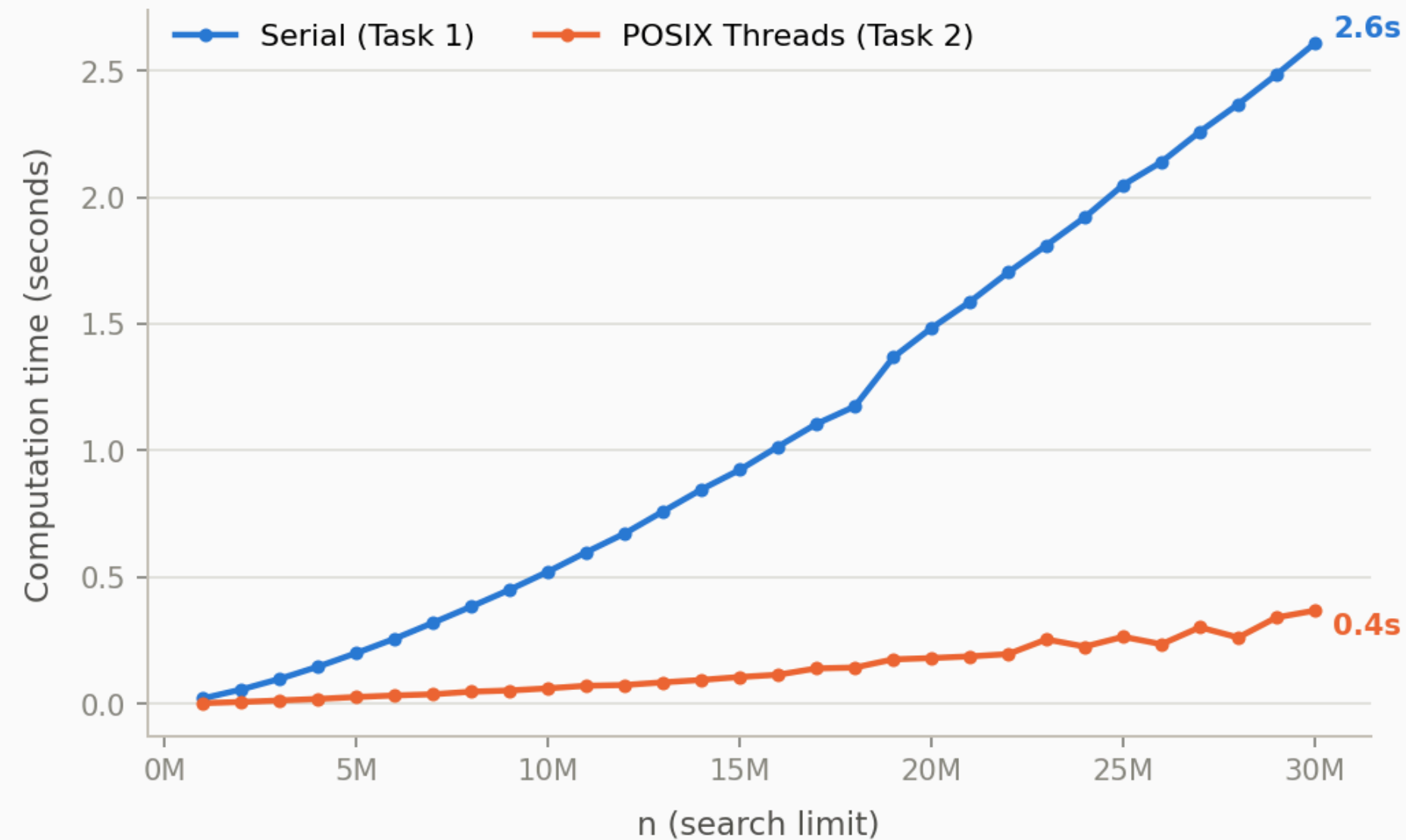
NO MUTEX

- Cost per number is uneven, big numbers take longer
- One block each, the top thread finishes last, 8.70x
- Chunks of 1000 dealt out in rotation, 9.75x
- Every thread gets a mix of cheap and costly numbers
- Each thread writes only its own slots, so no mutex

Serial vs POSIX Threads as n Grows

Graph 1: Run time, serial vs POSIX Threads, increasing n

14 threads. Lower is better.

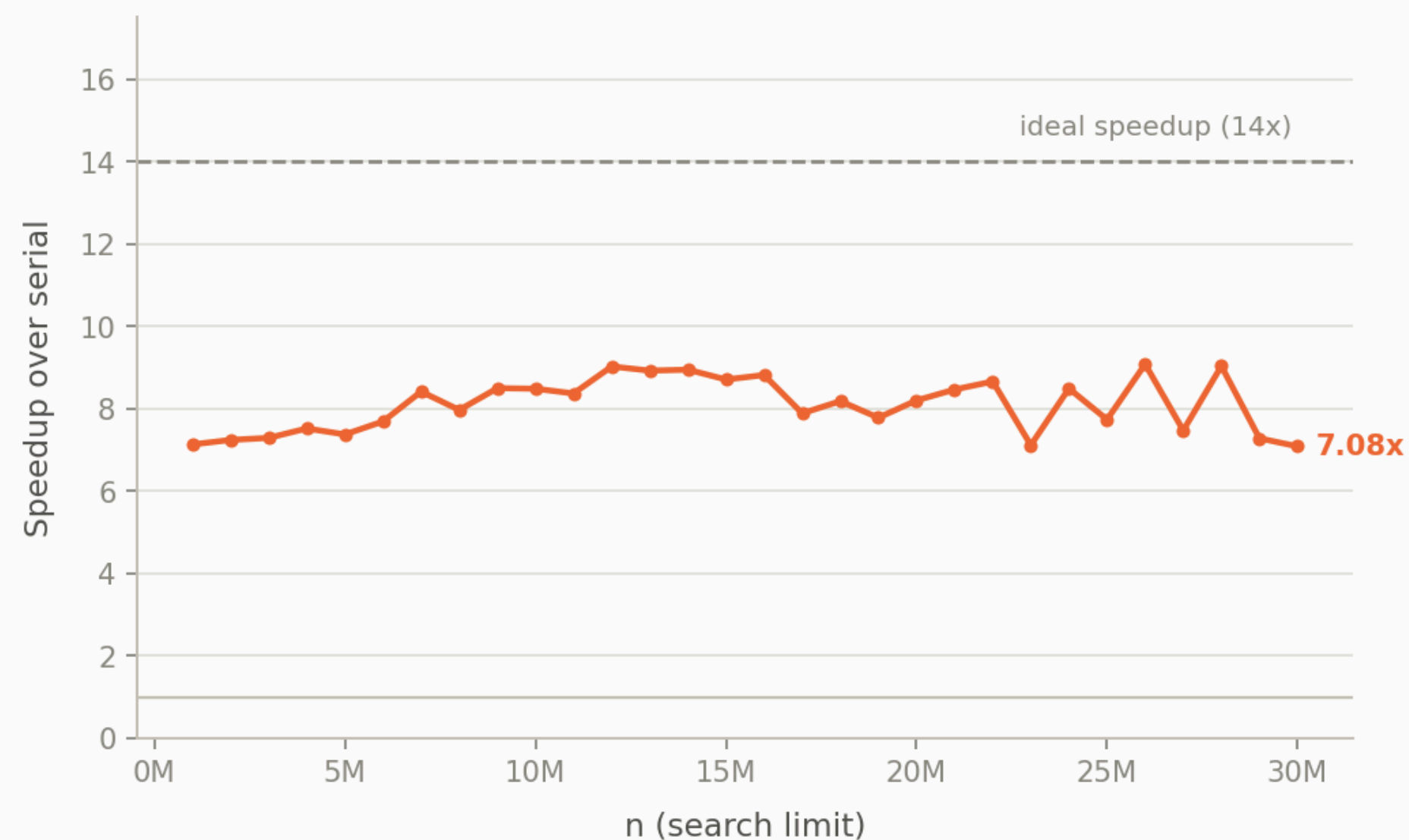


- Graph 1 of 8 required
- 30 values of n, from 1 million to 30 million
- Threads held fixed at 14
- Serial climbs from 0.02 s to 2.61 s
- Threads climb from 0.003 s to 0.37 s
- Gap widens as n grows, the work spreads over 14 cores

POSIX Threads Speedup as n Grows

Graph 2: Speedup of POSIX Threads, increasing n

14 threads. Speedup = serial time divided by parallel time.

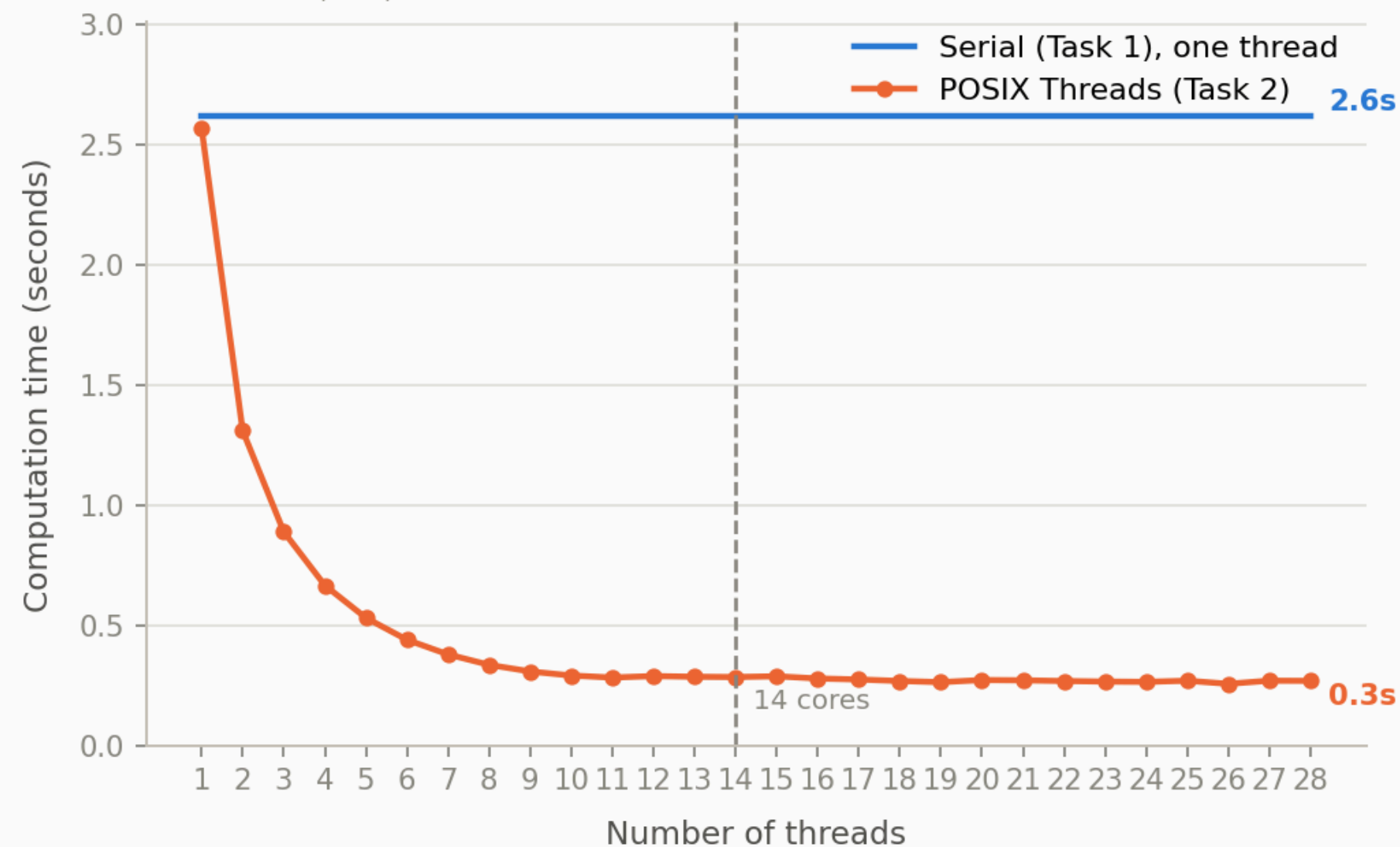


- Graph 2 of 8 required
- Speedup is serial time divided by parallel time
- Average 8.09x across all 30 runs
- Range 7.08x to 9.08x
- Holds steady, so it is not just a large n effect
- Dashed line is the ideal 14x, one per core

Serial vs POSIX Threads as Threads Grow

Graph 3: Run time, serial vs POSIX Threads, increasing threads

n held at 30,000,000. Lower is better.

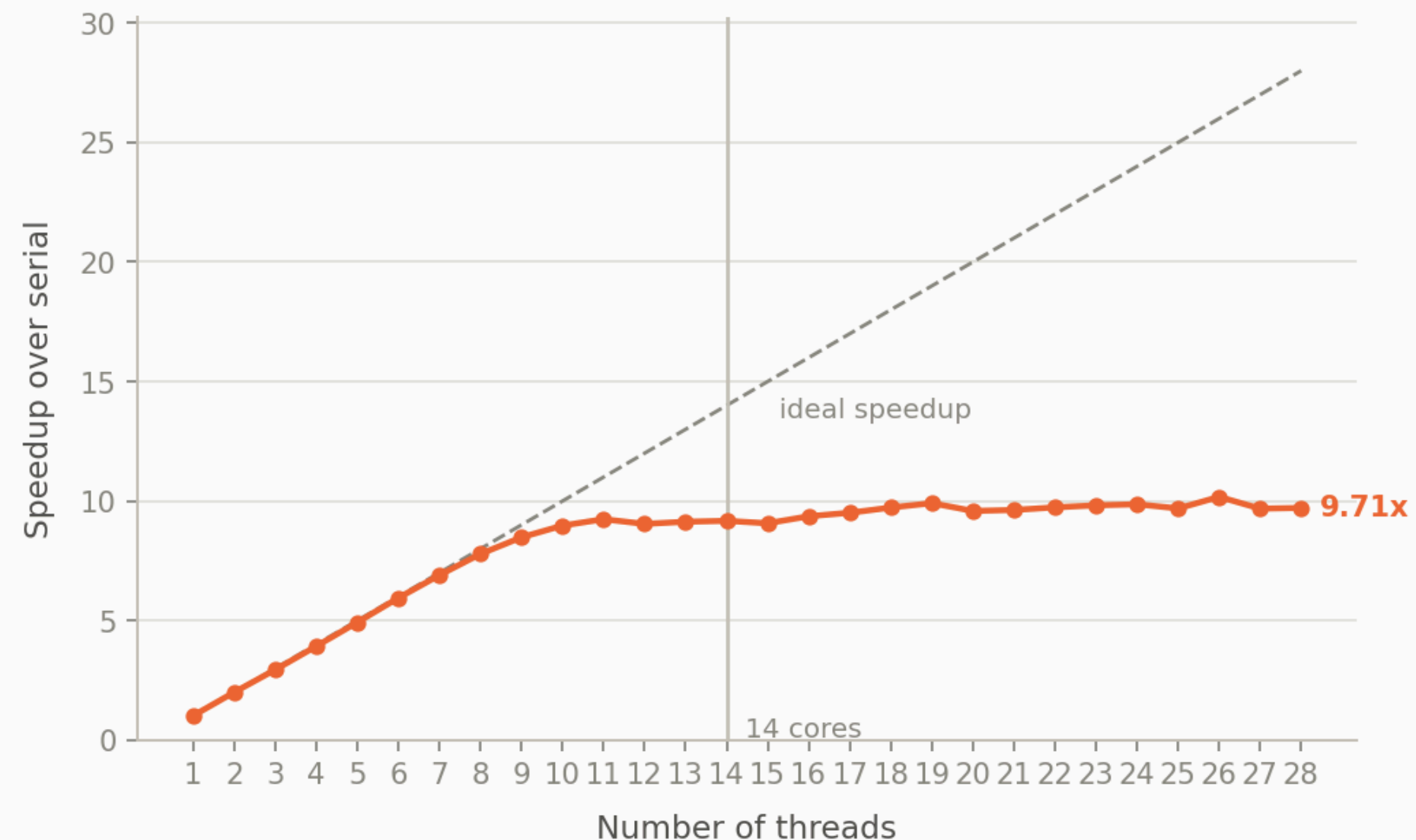


- Graph 3 of 8 required
- n now held fixed at 30 million
- Threads swept from 1 to 28, twice our core count
- Serial is flat at 2.62 s, it ignores thread count
- Threads fall from 2.57 s to 0.27 s
- Curve flattens once we pass 14 cores

POSIX Threads Speedup as Threads Grow

Graph 4: Speedup of POSIX Threads, increasing threads

n held at 30,000,000. Speedup stops improving once threads pass the core count.



- Graph 4 of 8 required
- 2 threads 2.00x, 4 threads 3.93x, 8 threads 7.79x
- Almost one to one up to 8 threads
- Plateaus near 9.7x once all 14 cores are busy
- Extra threads past 14 add nothing, they just take turns
- Not 14x because of Amdahl's Law and thread overhead
- 10 of our 14 cores are performance, 4 are efficiency

OpenMP, One Pragma, Same Parallelism

```
#pragma omp parallel for  
schedule(dynamic, 1000)
```

Compile with the fopenmp flag. Without it the pragma runs on one thread.

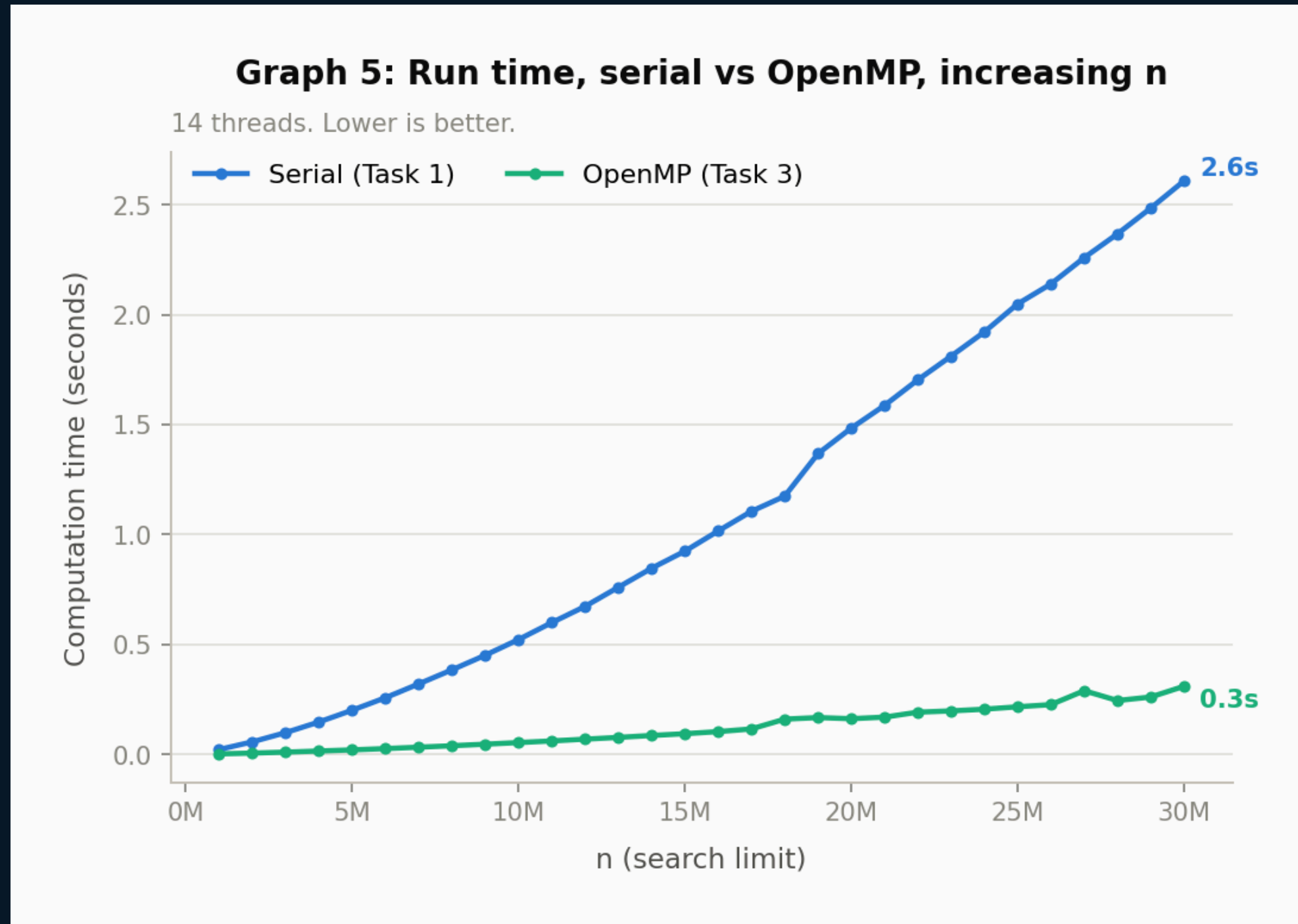
- One pragma replaces about 40 lines of thread code
- The loop body is completely unchanged
- We chose dynamic, our work gets heavier as n grows

~40 lines

of thread management replaced

STATIC, blocks decided before the loop starts
DYNAMIC, a thread takes the next free chunk
GUIDED, big chunks first, shrinking as it goes

Serial vs OpenMP as n Grows

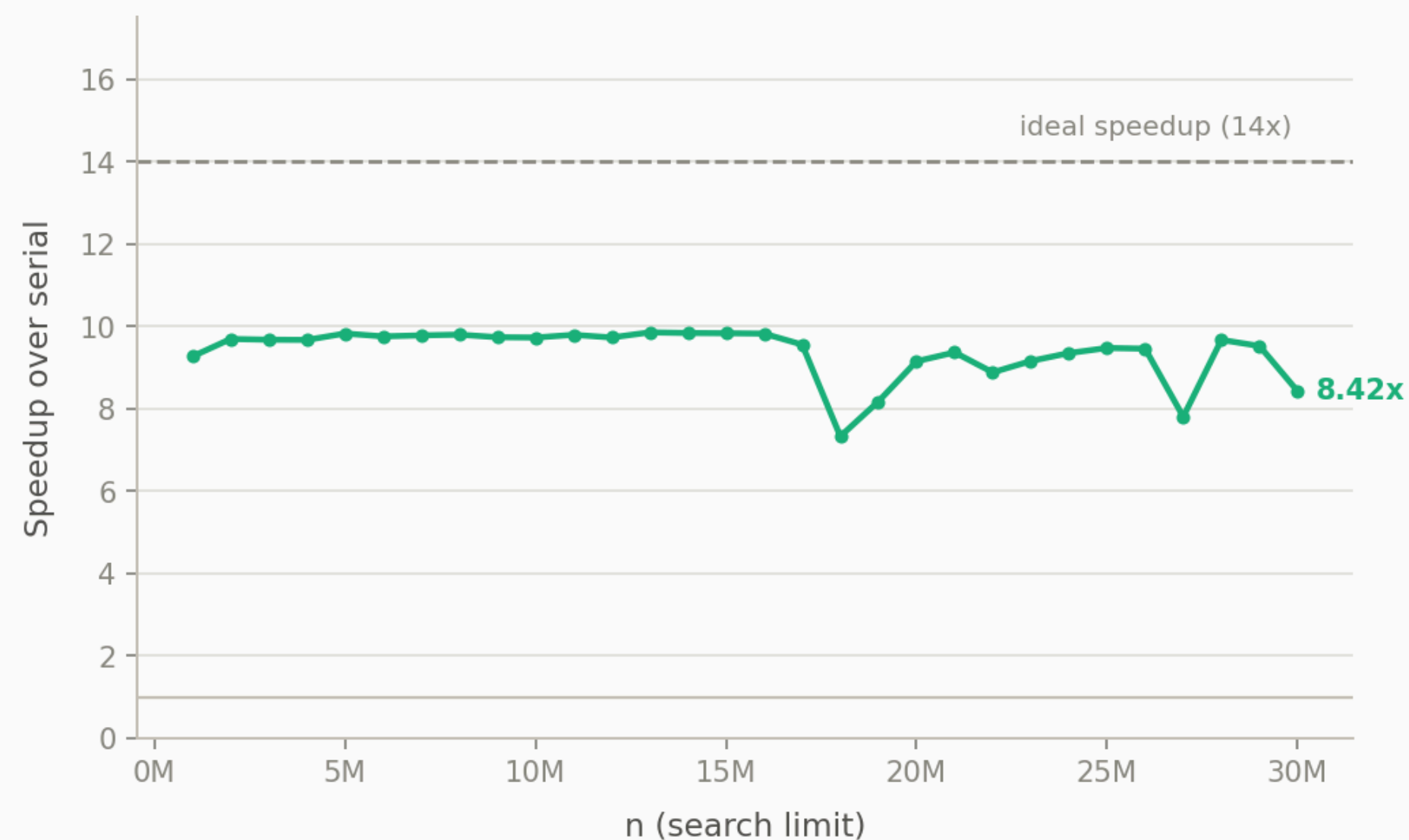


- Graph 5 of 8 required
- Same 30 values of n, same 14 threads
- OpenMP goes from 0.002 s to 0.31 s
- Serial goes from 0.02 s to 2.61 s
- Same shape as Graph 1, slightly lower throughout

OpenMP Speedup as n Grows

Graph 6: Speedup of OpenMP, increasing n

14 threads. Speedup = serial time divided by parallel time.

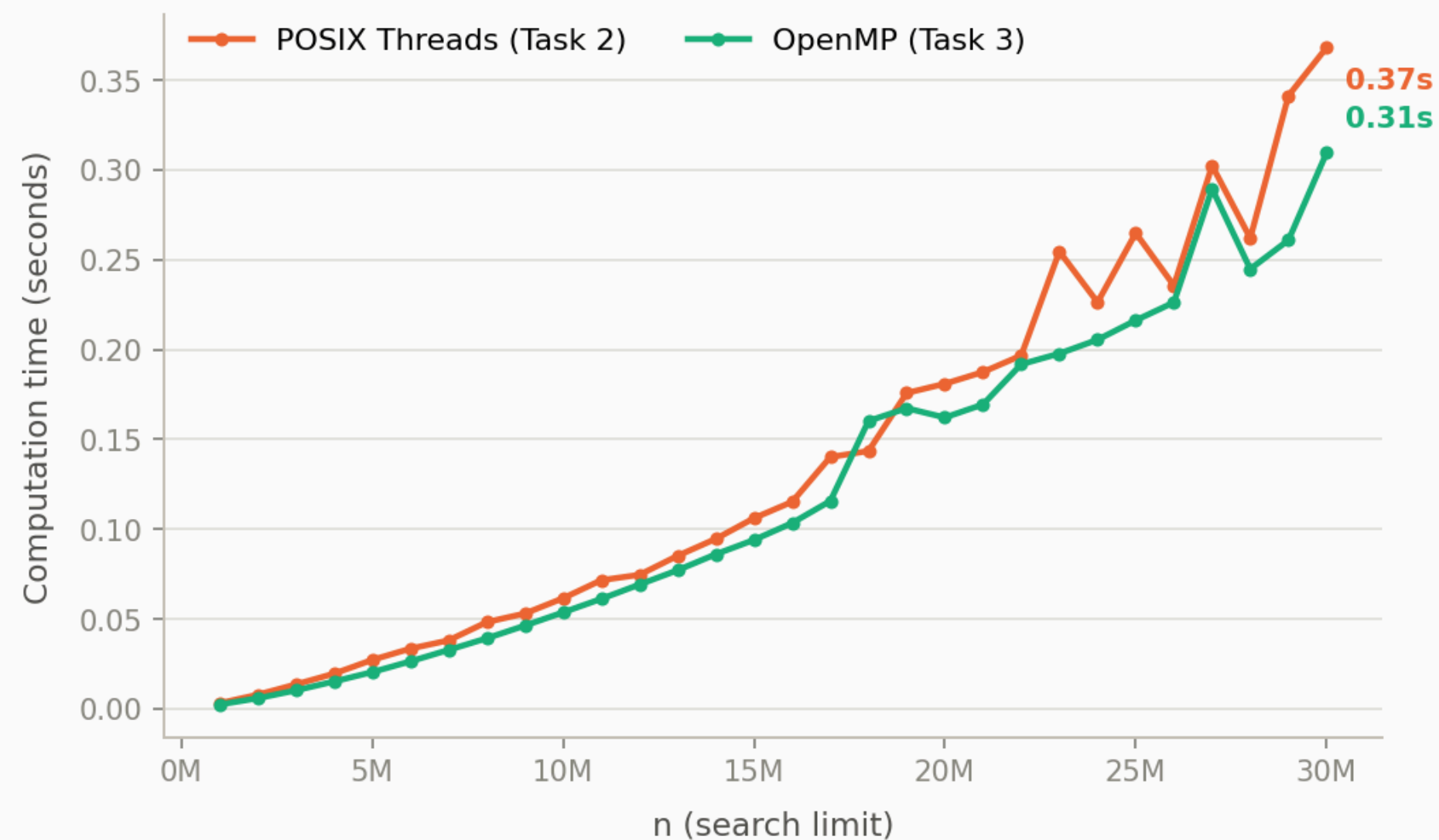


- Graph 6 of 8 required
- Average 9.37x across all 30 runs
- Range 7.33x to 9.85x
- Steadier than the POSIX Threads curve
- Dynamic scheduling rebalances while the loop runs
- Schedules measured at n = 30 million
static 9.55x, dynamic 10.44x, guided 10.42x

POSIX Threads vs OpenMP as n Grows

Graph 7: Run time, POSIX Threads vs OpenMP, increasing n

14 threads. Lower is better.

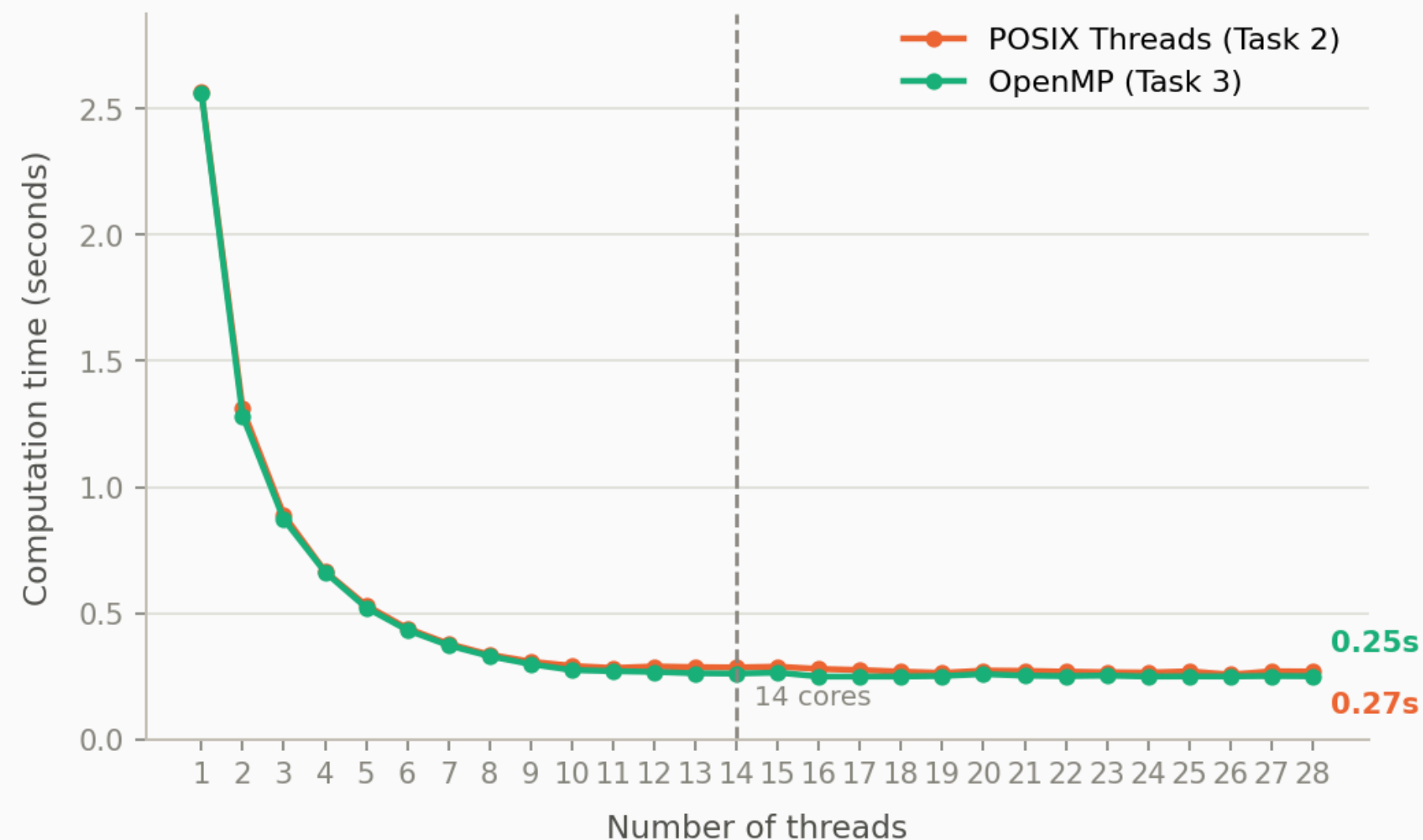


- Graph 7 of 8 required
- Both parallel versions on the same axes
- OpenMP faster at 29 of the 30 values of n
- At n = 30 million, 0.31 s against 0.37 s
- Same prime test, same chunk size, only sharing differs

POSIX Threads vs OpenMP as Threads Grow

Graph 8: Run time, POSIX Threads vs OpenMP, increasing threads

n held at 30,000,000. Lower is better.



- Graph 8 of 8 required
- n fixed at 30 million, threads 1 to 28
- Nearly identical up to 8 threads
- OpenMP pulls ahead once the cores fill up
- Peak 10.50x for OpenMP, 10.18x for POSIX Threads
- Dynamic adapts at run time, our rotation is fixed at launch
- OpenMP also reuses its thread team instead of making new ones

OpenMP Edges Ahead

8.09x

POSIX THREADS

Explicit creation, joins, chunk bookkeeping

9.37x

OPENMP

One pragma; compiler handles the team

faster on our machine • dramatically less code

What We Learned

01

Near linear speedup to 8 threads, then a plateau at the core count

02

Workload distribution decides everything, 8.70x became 9.75x

03

The flag array gave us sorted output and no mutex at once

04

More threads than cores gives nothing back

Limitation, trial division is not the fastest method. A segmented Sieve of Eratosthenes would scale far better and splits naturally by segment, at the cost of more memory and a harder implementation.

USE OPENMP

for simple loop parallelism

USE PTHREADS

when detailed thread control matters

Questions?

664,579

primes below 10 million

8.09×

POSIX Threads average

30

benchmark values of n

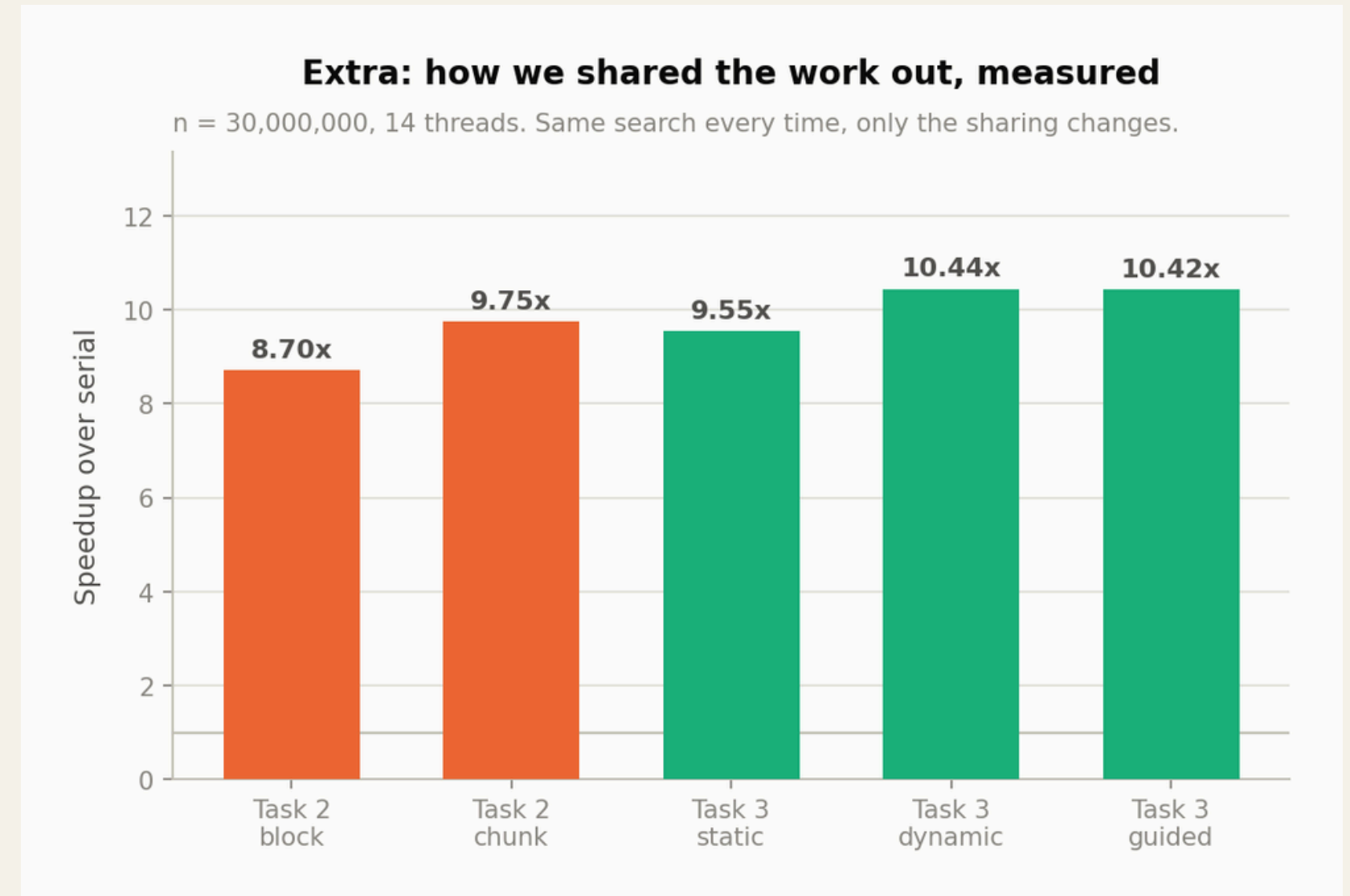
9.37×

OpenMP average

How the Numbers Were Measured

- 01 `run_benchmarks.sh`: build at 02, verify, then time
- 02 30 n values, threads 1 to 28, best of 3 runs
- 03 `python3 make_graphs.py` draws the eight required graphs plus the workload comparison

Apple M3 Max, 14 cores: 10 performance and 4 efficiency.
macOS 26.6.1. Timings exclude file output.



Chunks beat one block, 8.70x to 9.75x. Dynamic beats static by about 9 percent.

Thank you for your attention.

Erwyna Soo Wen Xin • 36555789
Taabish Farooq Bhat • 35473932

